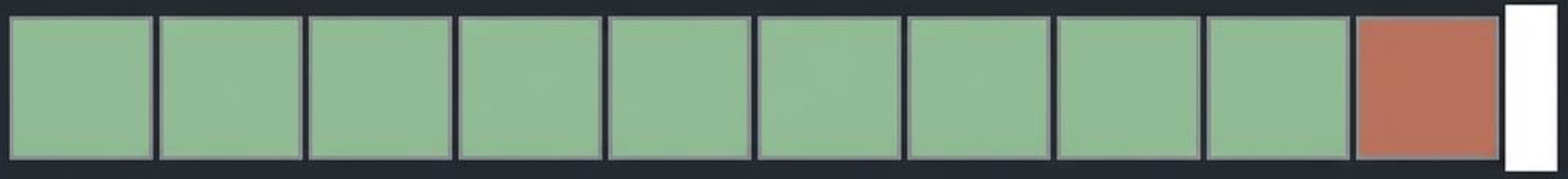
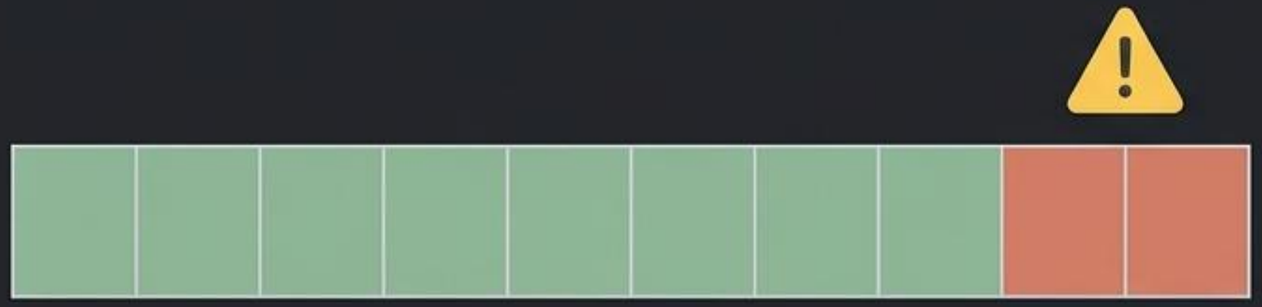
교차 검증(Cross-Validation)의 해부학



NumPy 엔진 직접 구현부터 Scikit-learn의 우아한 자동화까지

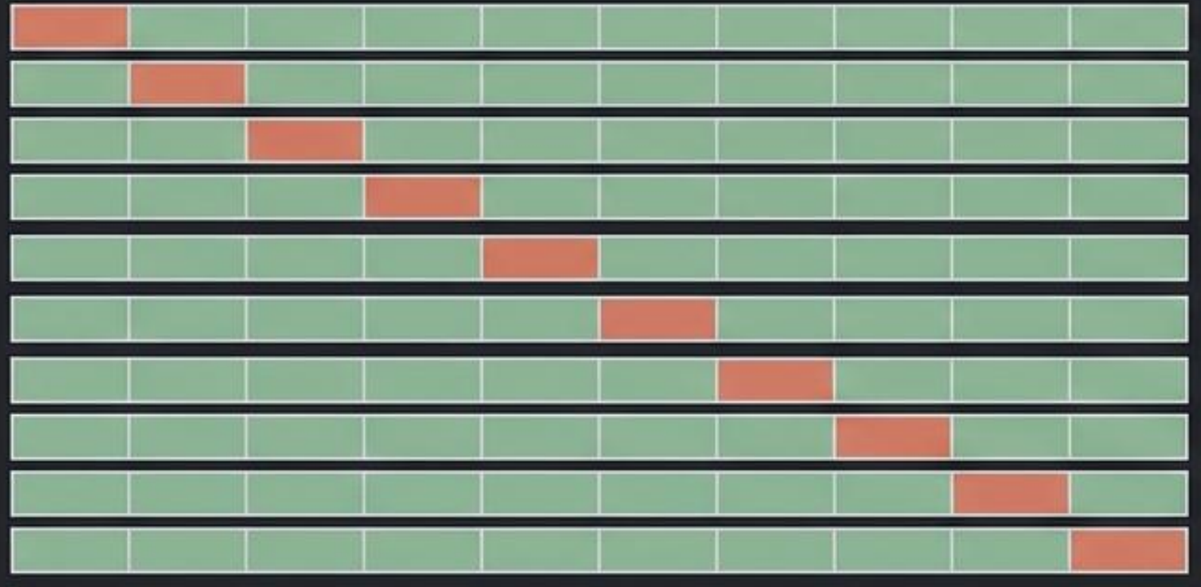
NotebookLM

단 한 번의 테스트로 모델을 신뢰할 수 있을까요?



데이터의 특정 부분에만 모델이
최적화되는 것을 방지해야 합니다.

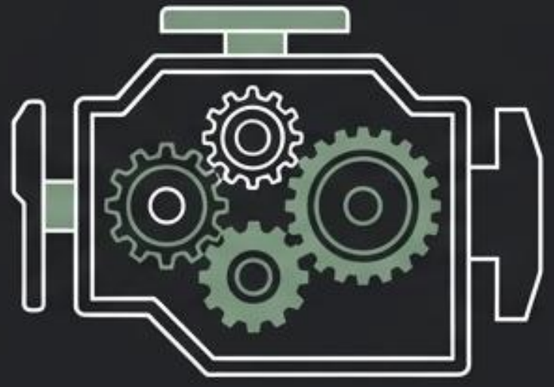
과적합(Overfitting)의 위험



10번의 교차 검증(10-Fold CV)으로
데이터의 모든 영역을 평가하여
확신을 만드세요.

NotebookLM

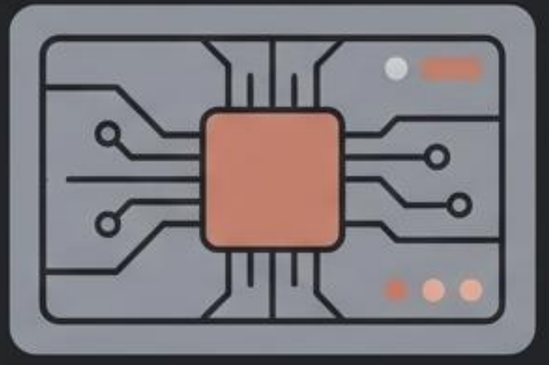
오늘 실습의 두 가지 경로



The Transparent Engine

Phase 1. 엔진 뜯어보기 (NumPy)

데이터 분할과 순환 루프의 내부 원리를
수동 코드로 직접 구현하며 이해합니다.



The Modern Dashboard

Phase 2. 모던 대시보드 (Scikit-learn)

실무 환경에서 사용하는 최적화된 함수로
복잡한 로직을 3줄로 압축합니다.

NotebookLM

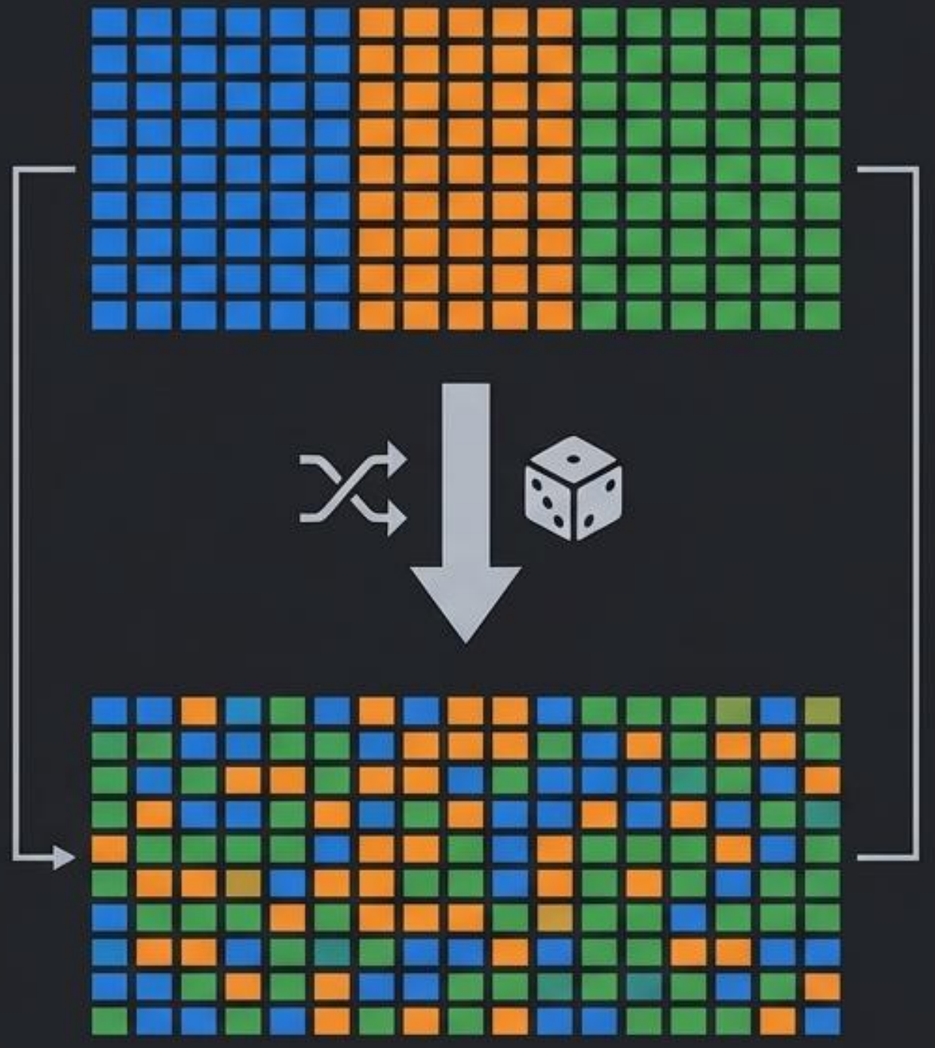
Step 1. 데이터 셔플 (The Crucial Shuffle)

> K-Fold 전 데이터 셔플은 선택이 아닌 필수입니다.
> (패턴 고착화 및 특정 Fold에 단일 클래스만 할당되는 현상 방지)

```
# 랜덤 결과를 고정 (재현성 확보)
np.random.seed(42)

# 데이터 인덱스를 랜덤하게 섞음
indices = np.random.permutation(len(x))

# 같은 순서로 X, y를 함께 섞어서 데이터-라벨 쌍 유지
x = x[indices]
y = y[indices]
```



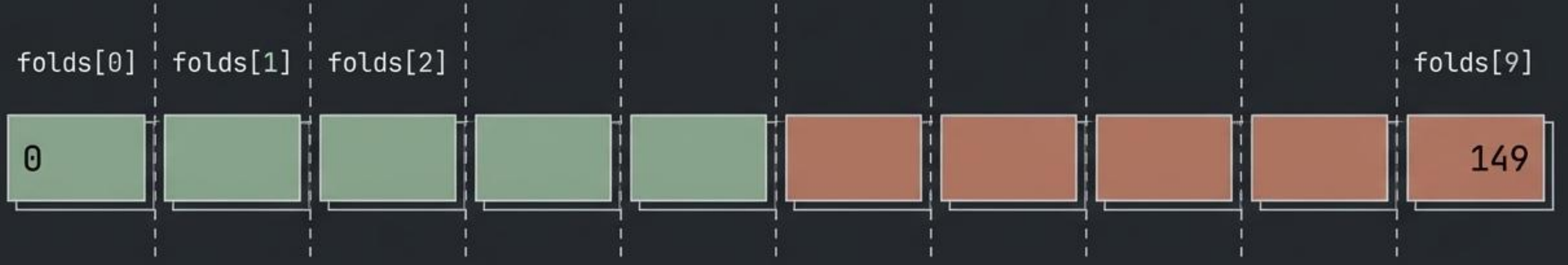
NotebookLM

Step 2. 인덱스 분할 (Slicing the Folds)

> 150개의 연속된 인덱스를 10개의 Fold 배열로 분리합니다.
> 이제 교차 검증을 위한 10개의 독립적인 테스트 티켓이 준비되었습니다.

```
# np.arange(len(x)) → [0, 1, 2, ..., N-1] 인덱스 생성
# np.array_split → 이를 10개로 나눔 (k=10)
folds = np.array_split(np.arange(len(x)), 10)

scores = []  # 각 fold의 accuracy 저장 리스트
```



NotebookLM

Step 3. 교차 검증 루프 (The Loop)

10번의 반복. 매번 다른 Fold가 Test Set이 되며, 모든 데이터가 정확히 한 번씩 평가받습니다.

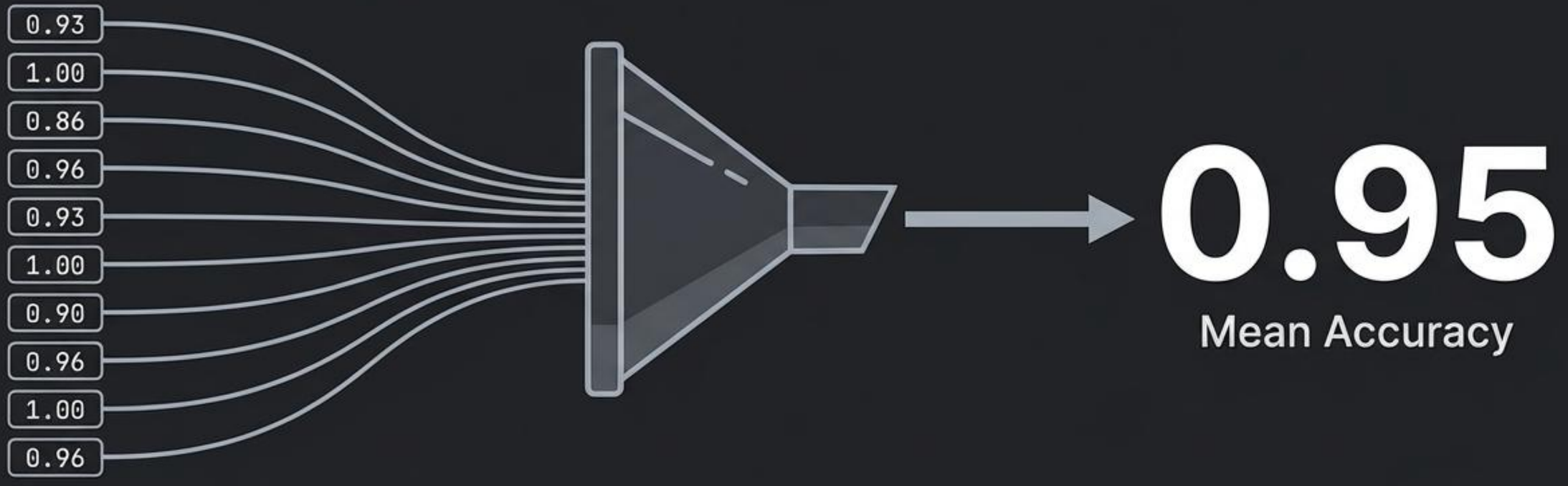



```
# k-fold cross validation 직접 구현
for j in range(10):
    # 현재 fold를 test set으로 사용
    test_indx = folds[j]
    # (이후 Train 세트 구성 및 모델 학습/평가 진행)
```

NotebookLM

Step 4. 전체 성능 산출 (Aggregating Results)

10번의 가혹한 개별 테스트 결과를 하나로 응집합니다. 이 평균 정확도(Mean Accuracy)야말로 미지의 데이터에 대한 모델의 '일반화 성능(Robustness)'을 나타내는 가장 신뢰할 수 있는 지표입니다.




```
# 10개 fold의 accuracy 평균 출력
print(np.mean(scores))
```

NotebookLM

실무에서도 이 엔진을 매번 조립해야 할까요?

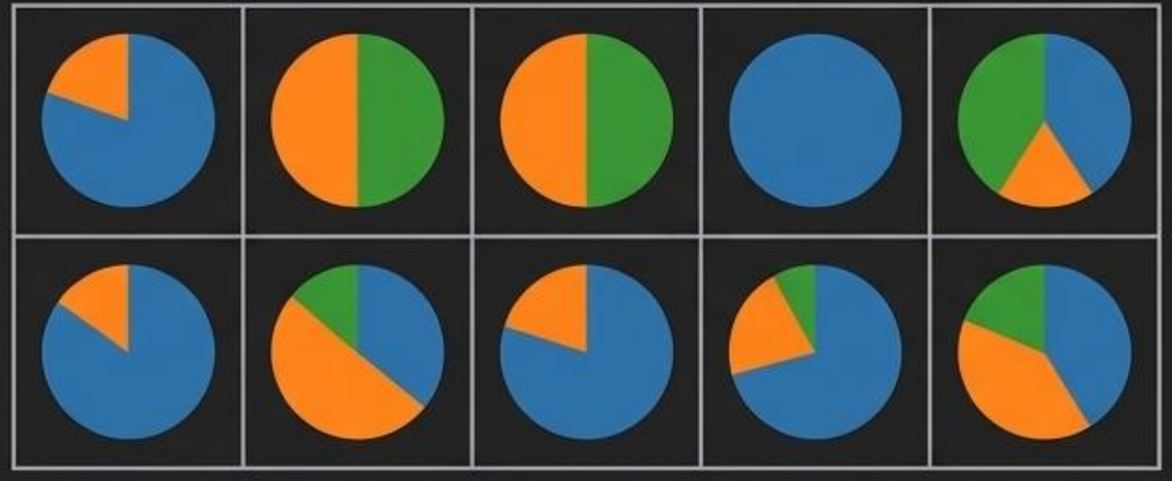
- 수동 제어는 원리 이해에 완벽하지만, 프로덕션 레벨에서는 휴먼 에러를 유발합니다.
- 바퀴를 재발명하지 마세요. 더 우아하고 안전한 도구가 존재합니다.

NotebookLM

Stratified K-Fold의 마법 (계층적 교차 검증)

무작위 셔플만으로는 우연히 특정 클래스가 쏠리는 현상을 막을 수 없습니다.
Stratified 방식은 모든 Fold가 원본 데이터의 클래스 비율을 정확히 유지하도록 강제합니다.

⚠️ 일반 Random K-Fold



🛡️ Stratified K-Fold



NotebookLM

단 3줄로 완성되는 모던 파이프라인



[모델 정의]

```
model = SVC(kernel='rbf')
```

평가할 머신러닝 알고리즘 객체 생성



[전략 정의]

```
cv = StratifiedKFold(n_splits=10, shuffle=True)
```

10-Fold 분할 및 클래스 편향 방지를 위한 계층화 셔플 옵션 적용



[실행 및 평가]

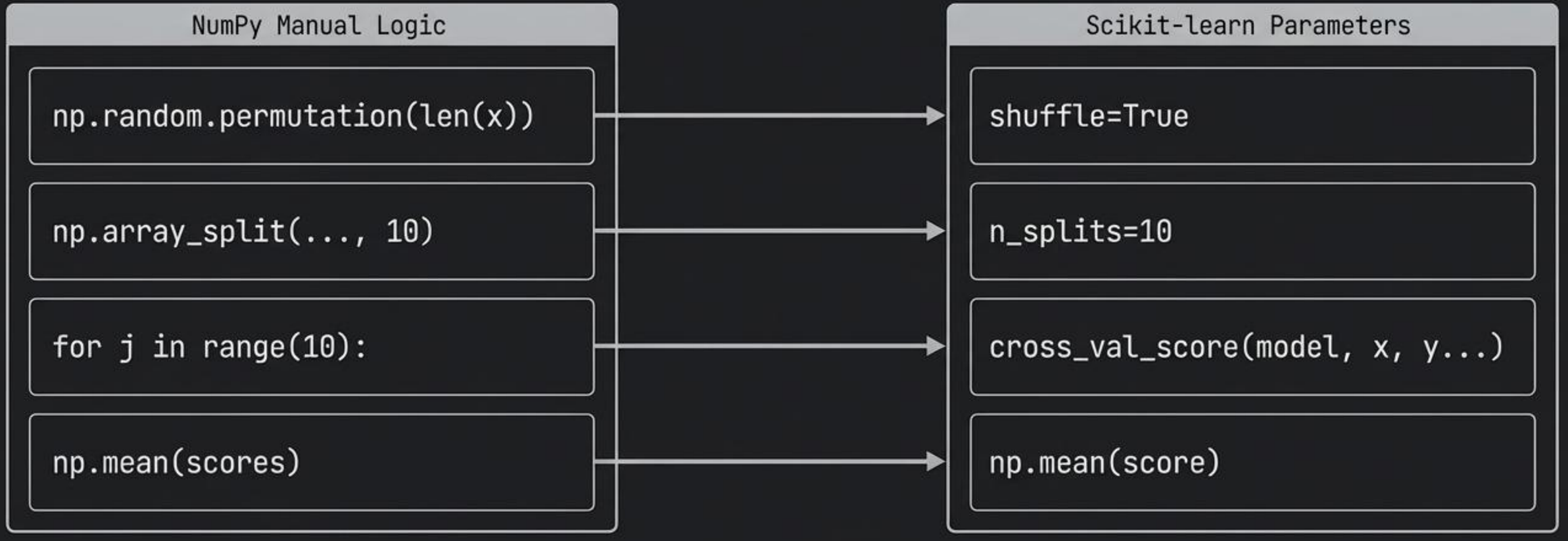
```
score = cross_val_score(model, x, y, cv=cv, scoring='accuracy')
```

데이터 분할, 루프 반복, 학습 및 정확도 산출까지 한 줄로 자동 실행

NotebookLM

두 세계의 연결 (Mapping the Logic)

수동 제어를 위해 작성했던 복잡한 로직들은 Scikit-learn 자동화 함수의 파라미터로 완벽히 치환됩니다.
엔진의 원리를 알면 대시보드를 100% 활용할 수 있습니다.



NotebookLM

언제 무엇을 써야 할까? (Comparison Matrix)

	NumPy 수동 구현	Scikit-learn 자동화
코드 & 유지보수	길고 복잡함. 휴먼 에러 발생 가능성 높음.	매우 짧고 명료함. 표준화되어 유지보수 용이.
클래스 불균형 제어	별도의 복잡한 수동 로직 추가 필요.	`StratifiedKFold`로 완벽하고 간단하게 지원.
최적의 사용 시기	새로운 자작 알고리즘 테스트, 딥러닝 커스텀 루프 작성, 교육 목적.	일반적인 머신러닝 프로덕션 환경, 표준 모델 평가.

NotebookLM

교차 검증의 3가지 핵심 기둥



Shuffle
(데이터 무작위 혼합)



K-Fold
(독립적인 분할과 순환 평가)



Stratified
(클래스 비율의 완벽한 유지)

```
print(np.mean(score))
```

출력된 평균 정확도(Mean Accuracy)는 단순한 하나의 숫자가 아닙니다.
10번의 혹독한 검증 루프를 통과한, 여러분의 모델이 증명해낸 신뢰의 크기입니다.

NotebookLM